

ChargeEV

ChargeEV — Complete Codebase Summary

An EV charging station management platform. Full-stack JavaScript, with real-time auctions, Stripe payments, and role-based access for drivers, station owners, and admins.

This document is the single-page orientation for the whole project. For deeper detail, see the reference docs listed in §9.

1. At a glance

Frontend	React 18 + Vite + Redux Toolkit + Bootstrap 5
Backend	Express + Prisma + MongoDB + Socket.IO + Stripe
Source size	~14,600 lines frontend · ~6,900 lines backend
Files	45 <code>.jsx</code> , 24 <code>.js</code> , 2 <code>.css</code> (frontend) · 60 <code>.js</code> (backend)
Pages	26 (20 public/user + 6 admin)
API	62 route handlers across 13 route files
Data model	10 Prisma models, 6 enums
Tests	73 passing across 5 suites
Deployment target	Vercel (frontend) + Render (backend) + MongoDB Atlas

2. Architecture

```
ev-management/
├─ frontend/                React 18 SPA (Vite)
│  └─ src/
│     ├─ pages/             26 route-level pages (incl. pages/admin/)
│     ├─ components/        16 shared components
│     ├─ store/             Redux Toolkit – 13 slices + cache layer
│     ├─ utils/             9 helpers (api, socket, logger, analytics...)
│     └─ styles/            grid.css (staged for Phase 10)
│
├─ backend/                Express REST API + Socket.IO
│  ├─ controllers/          11 controllers (business logic)
│  ├─ routes/              13 route files (62 handlers)
│  ├─ middleware/           auth, error, kyc, validate, validateQuery
│  ├─ utils/               15 helpers (prisma, jwt, email, stripe...)
│  ├─ prisma/              schema.prisma + seed.js
│  ├─ tests/               5 suites, 73 tests (in-memory mock DB)
│  └─ scripts/             doctor.mjs (preflight check)
│
├─ docs/screenshots/        Design mockups
└─ scripts/                setup-env.mjs (generates .env with secrets)
```

Request flow: React page → Redux thunk → `utils/api.js` (axios) → Express route → validation middleware → auth/KYC middleware → controller → Prisma → MongoDB. Real-time updates (auctions, bids, notifications) bypass this and push over Socket.IO.

3. Data model (Prisma / MongoDB)

Models: `User`, `EV`, `ChargingStation`, `Slot`, `Booking`, `Review`, `Bid`, `Payment`, `Log`, `Complaint`

Enums: `Role`, `StationStatus`, `SlotStatus`, `BookingStatus`, `BidStatus`, `PaymentStatus`

Core relationships: a `User` (driver) owns `EV` s and makes `Booking` s against a `Slot` belonging to a `ChargingStation` owned by another `User` (owner). Contested slots go to auction via `Bid` . Completed bookings unlock `Review` s. `Payment` records tie to Stripe. `Log` powers the admin audit trail.

4. Roles & permissions

Role	Can do
Driver	Register EVs, browse/book stations, bid in auctions, pay, review, file complaints

Owner	Everything above + create stations & slots (pending admin approval), view revenue reports, close auctions
Admin	Approve/reject stations, manage users, resolve complaints, view dashboard + full audit log

Enforced by `auth.middleware.js` (JWT) and `kyc.middleware.js` (gates booking/listing behind identity verification).

5. API surface (62 endpoints)

Route file	Endpoints	Covers
<code>admin</code>	9	Dashboard, user/station management, logs
<code>auth</code>	8	Register, login, logout, password reset, admin setup
<code>booking</code>	7	Create, check-in, complete, cancel, history
<code>slot</code>	7	Slot CRUD, availability
<code>station</code>	7	Station CRUD, public listing, approval
<code>bid</code>	5	Place bid, auction close, priority scoring
<code>ev</code>	5	User's EV CRUD
<code>verification</code>	4	Email OTP, KYC
<code>complaint</code>	3	Submit, admin inbox, resolve
<code>review</code>	3	Verified-purchase reviews
<code>payment</code>	2	Stripe intent + webhook
<code>ai</code>	1	Station recommendation
<code>user</code>	1	Profile

Full request/response reference: `API.md`.

6. Notable engineering decisions

These came up repeatedly and are worth knowing before changing things:

- **Auction priority scoring** = 60% normalized bid + 40% battery urgency, normalized against a *fixed* reference ceiling (not the running max bid) — so rankings stay stable and order-independent regardless of bid sequence.
- **Booking race safety** — concurrent booking attempts on the same slot are rejected correctly under real parallel load (verified with a `Promise.all` test, not sequential calls).
- **Stripe webhook idempotency** — a replayed webhook charges exactly once.
- **Caching without new infrastructure** — the stack constraint ruled out React Query and Redis, so: a ~30-line TTL cache using Redux Toolkit's own `createAsyncThunk` `condition` (frontend) and a `Map`-based TTL cache (backend). Both deliberately scoped to low-risk targets only — never applied to pages with live Socket.IO or admin correctness needs.
- **Email XSS hardening** — user-supplied names are auto-escaped via a tagged template across all 12 email templates.
- **Dev-gated logging** — all logging routes through `logger.js` on both sides; `logger.debug` is dev-only, so routine events (like an expired JWT) don't create production noise.
- **Graceful boot failures** — the server verifies the DB with a `SELECT 1` before listening, and `npm run doctor` diagnoses env/client/DB problems with specific fixes rather than a stack trace.

7. What the engagement covered (Phases 1-10)

Phase	Outcome
1	Structure audit, dev-gated logger, PropTypes on 18 components, env setup script
2	Full rebrand — cream/gold/dark palette (<code>#FDF8F0</code> / <code>#C9A96E</code> / <code>#1A1A1A</code>)
3	CTA audit, 44px touch targets, empty-state CTAs
4	SEO — meta tags on all 26 pages, JSON-LD, sitemap, heading hierarchy
5	Feature verification, dead-code report, silent-failure bug fixes, mobile audit
6	Performance — WebP, self-hosted fonts, DB indexes, fixed a public endpoint leaking revenue data

7	Testing/QA — race-condition test, stored-XSS fix, WCAG contrast math, Modal ARIA rebuild, 58 form-label fixes
8.1	Docs — README, <code>API.md</code> (65 endpoints), deployment guide
8.2	Code quality — ESLint + Prettier from scratch, console.log audit, naming conventions
8.3	Deployment prep — real health check, Sentry, Google Analytics 4
9	Final verification — fixed 3 crashing test suites , full checklist audit
10	Bootstrap removal — scoped and staged (deliberately paused, see §8)

15 git commits, all logically scoped. Running log: `CHANGELOG.md`.

8. Current status

All automated checks pass: 73/73 tests · ESLint clean both sides · frontend build succeeds · 313/313 imports resolve case-exactly · 0 backend vulnerabilities · no TODO/FIXME markers in source.

Known open items (deliberate, not bugs): 1. **Bootstrap removal is paused mid-plan** — `grid.css` is written but *not imported*, and Bootstrap is still fully active. The app is in a consistent state; the migration simply hasn't touched any page yet. Resume via the pilot-page approach in `PHASE_10_BOOTSTRAP_REMOVAL_PLAN.md`. 2. **DRY / component-extraction pass** — last unfinished Phase 8.2 item. 3. **Sentry + GA are wired but dormant** — they no-op until you supply `SENTRY_DSN` / `VITE_GA_MEASUREMENT_ID`. 4. **Optional Dockerfile** — not created (Vercel/Render don't need it). 5. **Frontend esbuild / vite advisory** — dev-server only, needs a breaking Vite major bump, intentionally deferred.

Two things this environment could never verify (no live browser, no real database): anything visual/browser-side (console errors, real 404 navigation, mobile rendering, design polish), and a live backend boot against real MongoDB. Both are covered in `CURRENT_STATE_REPORT.md`.

9. Where to look next

Doc	Use it for
CURRENT_STATE_REPORT.md	Bugs/gaps/functionality status — start here
README.md	Setup, tech stack, design, security
DEPLOYMENT.md	Vercel + Render + Atlas, step by step
API.md	Every endpoint, exhaustively
CHANGELOG.md	Full engagement history, most recent first
PHASE_10_BOOTSTRAP_REMOVAL_PLAN.md	Resuming the CSS migration
PHASE_9.2_FINAL_REPORT.md	Prior summary + recommendations
PHASE_7.1_TEST_PLAN.md	What's live-tested vs. code-traced
PROJECT_SUMMARY.md , SETUP_GUIDE.md	Original pre-engagement docs

10. Getting it running

```

node scripts/setup-env.mjs      # generates both .env files with real secrets
# then fill in backend/.env: DATABASE_URL (required), SENDGRID/STRIPE/SENTRY (optional)

cd backend && npm install
npm run doctor                  # preflight: env, Prisma client, DB connectivity
npm start                       # → http://localhost:5000

cd frontend && npm install
npm start                       # → http://localhost:3000

```

Verify anytime: `cd backend && npm test` → expect **73/73 passing**.